LAB 4: EXERCISES

Các kiểu dữ liệu nâng cao trong C và thuật toán sắp xếp

Lập trình hệ thống (UET.CE2020)
Instructor: Pham Van Duc

Ngày 9 tháng 10 năm 2026

Bài 1: Quản lý tọa độ robot

Mô tả bài toán: Một robot di chuyển trong mặt phẳng 2D và được biểu diễn bằng tọa độ (x, y). Chương trình cần lưu và cập nhật vị trí robot.

Yêu cầu:

- Khai báo struct `Point` gồm hai thành phần x, y.
- Nhập tọa độ ban đầu của robot.
- Viết hàm cập nhật vị trí.
- In vị trí sau khi cập nhật.

Ví dụ: Input: `(3,4)`, dịch chuyển `(2,1)` $\rightarrow$ Output: `(5,5)`

Bài 2: Quản lý sinh viên

Mô tả bài toán: Một lớp học cần quản lý thông tin sinh viên gồm mã sinh viên, họ tên, tuổi và điểm số. Chương trình cần cho phép nhập danh sách sinh viên, tìm kiếm, cập nhật, thống kê và sắp xếp dữ liệu.

Yêu cầu:

- Sử dụng `typedef` để định nghĩa kiểu dữ liệu `Student`.
- Khai báo kiểu `Student` gồm `id`, `name`, `age`, `score`.
- Nhập số lượng sinh viên và nhập danh sách sinh viên.
- In danh sách sinh viên.
- Tìm kiếm sinh viên theo `id`.
- Nếu tìm thấy, in thông tin sinh viên tương ứng.
- Cập nhật điểm cho sinh viên theo `id`.
- Tính điểm trung bình của cả lớp.
- Tìm sinh viên có điểm cao nhất.

- Sắp xếp danh sách theo điểm giảm dần.
- Sắp xếp danh sách theo tên theo thứ tự bảng chữ cái.
- In danh sách sau khi sắp xếp.

Bài 3: Hệ thống tài khoản ngân hàng

Mô tả bài toán: Một tài khoản ngân hàng có số dư và cần thực hiện các thao tác nạp và rút tiền.

Yêu cầu:

- Khai báo struct `Account` gồm `id`, `balance`.
- Viết hàm `deposit(Account *acc, int amount)`.
- Viết hàm `withdraw(Account *acc, int amount)`.
- Cập nhật trực tiếp số dư thông qua con trỏ.
- Không cho phép rút nếu số dư không đủ.

Bài 4: Hệ thống thu thập dữ liệu cảm biến

Mô tả bài toán: Một hệ thống nhúng thu thập dữ liệu từ cảm biến nhiệt độ theo thời gian. Mỗi lần đo gồm giá trị nhiệt độ, thời điểm đo và trạng thái cảm biến. Chương trình cần lưu trữ, hiển thị và xử lý dữ liệu cảm biến.

Yêu cầu:

- Sử dụng `typedef` để định nghĩa kiểu dữ liệu `SensorData`.
- Khai báo một kiểu `enum` để biểu diễn trạng thái cảm biến, ví dụ: `NORMAL`, `WARNING`, `ERROR`.
- Khai báo struct `SensorData` gồm:
  - `float temperature`
  - `int timestamp`
  - Trạng thái cảm biến
- Nhập danh sách dữ liệu cảm biến.
- In toàn bộ dữ liệu cảm biến.
- Tính nhiệt độ trung bình.
- Tìm giá trị nhiệt độ lớn nhất.
- Đếm số lần cảm biến ở từng trạng thái.
- Sắp xếp dữ liệu theo thời gian tăng dần.
- Sử dụng `qsort()` để thực hiện việc sắp xếp.
- Viết hàm so sánh phù hợp để truyền vào `qsort()`.

Bài 5: Struct với cấp phát động

Mô tả bài toán: Dữ liệu trong struct có kích thước không cố định và cần cấp phát động.

Yêu cầu:

- Khai báo struct `Student` gồm:
  - `char *name`
  - `int age`
- Dùng `malloc()` để cấp phát bộ nhớ cho tên.
- Nhập và in thông tin sinh viên.
- Giải phóng bộ nhớ sau khi sử dụng.
- So sánh shallow copy và deep copy.

Bài 6: Union lưu dữ liệu đo

Mô tả bài toán: Một thiết bị đo có thể trả về dữ liệu dưới các dạng khác nhau, ví dụ số nguyên hoặc số thực, nhưng tại một thời điểm chỉ có một giá trị là hợp lệ. Chương trình cần sử dụng union để lưu dữ liệu đo và hiển thị kết quả tương ứng.

Yêu cầu:

- Khai báo `union` để lưu một trong hai kiểu dữ liệu: `int` hoặc `float`.
- Viết chương trình cho phép người dùng chọn kiểu dữ liệu cần nhập.
- Nếu người dùng chọn số nguyên, lưu và in giá trị nguyên.
- Nếu người dùng chọn số thực, lưu và in giá trị thực.
- In kích thước của `union` bằng `sizeof()` và nhận xét.

Bài 7: Danh sách liên kết đơn

Mô tả bài toán: Xây dựng danh sách liên kết để quản lý dữ liệu động.

Yêu cầu:

- Khai báo struct node.

```
struct Node {
    int data;
    struct Node *next;
};
```

Yêu cầu:

- Tạo node.
- Thêm node vào danh sách.
- Duyệt và in danh sách.